Netflix Movie Recommendation System: Technical Report

---

1. Problem Understanding

In the era of infinite digital content, discovering relevant movies is a significant challenge for users. Streaming platforms rely heavily on recommendation systems to drive user engagement and retention. The primary objective of this project is to develop a personalized recommendation engine using historical user-item interaction data from the Netflix Prize Dataset. The core challenges involve handling massive dataset sizes, mitigating extreme data sparsity (where most users have rated only a tiny fraction of available content), and solving the computational bottlenecks associated with large-scale collaborative filtering.

2. Exploratory Data Analysis (EDA)

The original Netflix Prize dataset consists of over 100 million ratings across 480,189 users and 17,770 movies. Our EDA revealed a strong power-law distribution in user activity and movie popularity:

- **User Activity:** The median user rated 96 movies, while the 99th percentile of power-users rated up to 1,390 movies.
- **Movie Popularity:** Blockbusters heavily skew the data. The Top-10 movies (e.g., Pirates of the Caribbean, Forrest Gump) received nearly 50,000 ratings each, while thousands of niche titles received very few.
- **Data Sparsity:** The raw interaction matrix exhibits over 98% sparsity.

To improve model stability and computational efficiency, intelligent filtering was applied. By retaining only users with ≥500 ratings and movies with ≥1000 ratings, the dataset was reduced to 45.2 million ratings (54,404 users, 7,127 movies), yet it still maintained an 88.32% sparsity rate.

3. Methodology

Given the 88.32% sparsity, traditional memory-based Collaborative Filtering (e.g., User-User KNN) is mathematically and computationally infeasible due to the $O(N^2)$ memory requirement for similarity matrices. Instead, Model-Based Collaborative Filtering via Latent Factor Models was employed. Specifically, Neural Recommendation Architectures were utilized to project users and movies into lower-dimensional dense embedding spaces, allowing the models to capture complex, non-linear latent interactions.

Train / Test Split

A random 90/10 holdout split was used. 90% of all (user, movie, rating) triplets were used for training; the remaining 10% were held out strictly for evaluation. The split was performed at the interaction level (not at the user level), ensuring that both sets contain ratings from the same pool of users and movies. A fixed random seed was set to guarantee reproducibility.

4. Model Design

Two distinct architectures were designed and evaluated using PyTorch:

A. Generalized Matrix Factorization (GMF)

- **Architecture:** Learns dense embeddings of size 128 for both users and items. The interaction is modeled as a simple dot-product between the vectors.

- **Components:** User Embedding (128d), Item Embedding (128d), User Bias (scalar), Item Bias (scalar), Global Mean (scalar).
- **Parameters:** ~7.9 Million.

B. Deep Neural Collaborative Filtering (Deep NCF)

- **Architecture:** Concatenates user and item embeddings and feeds them through a Multi-Layer Perceptron (MLP) to learn highly non-linear interaction functions.
- **Components:** User Embedding (64d), Item Embedding (64d), MLP Layers (256 → 128 → 64 → 1), ReLU activations, 20% Dropout.
- **Parameters:** ~4.0 Million.

5. Evaluation Metrics

The models were evaluated using the following mandatory metrics on a 10% holdout test set:

- **RMSE (Root Mean Squared Error):** Measures rating prediction accuracy by penalizing large prediction errors.
- **MAP@10 (Mean Average Precision at 10):** Measures recommendation ranking quality.

Relevance Definition

A recommended movie was considered relevant to the user if the ground-truth rating in the test set was ≥3.5 stars.

MAP@10 Computation Procedure

- Step 1 — For each user in the test set, identify all movies the user has not rated in the training set.
- Step 2 — Use the trained model to predict ratings for all candidate movies, then rank them in descending order.
- Step 3 — Keep the top 10 ranked movies as the recommendation list.
- Step 4 — For each position k in the Top-10, check whether the movie appears in the user's test set and has a ground-truth rating ≥3.5.
- Step 5 — AP@10 per user = (1 / |Rel|) × Σ [P@k × rel(k)], where |Rel| is the number of relevant items in the test set for that user.
- Step 6 — MAP@10 = average of AP@10 values across all users. Only users with at least one relevant item are included.

6. Experimental Results

Both models were trained using the AdamW optimizer and a OneCycle Learning Rate Scheduler.

Model	Parameters	Train RMSE	Test RMSE	MAP@10
GMF	7.9M	0.7152	0.7717	0.1673
Deep NCF	4.0M	0.8126	0.8123	0.0743

Result Analysis: The target RMSE of ~0.80 was successfully achieved. GMF significantly outperformed Deep NCF. The deep non-linear layers in the NCF model struggled to generalize over the sparse user-item matrix compared to the elegant, mathematically constrained dot-product of Matrix Factorization.

7. Recommendation Examples

An inference pipeline was built to generate personalized Top-10 movie recommendations. For User ID 712664 (a user who historically rated sci-fi/fantasy highly), the GMF model recommended:

1.	Lord of the Rings: The Return of the King	Predicted: 5.0/5
2.	Lord of the Rings: The Fellowship of the Ring	Predicted: 5.0/5
3.	Star Wars: Episode V	Predicted: 5.0/5
4.	The Matrix	Predicted: 4.8/5
5.	Star Wars: Episode IV	Predicted: 4.8/5
6.	Indiana Jones and the Last Crusade	Predicted: 4.7/5
7.	Lord of the Rings: The Two Towers	Predicted: 4.7/5
8.	Gladiator	Predicted: 4.6/5
9.	Braveheart	Predicted: 4.6/5
10.	Schindler's List	Predicted: 4.5/5

Success Cases: The model successfully clusters similar latent tastes, accurately identifying the user's affinity for high-budget fantasy franchises.

Failure Cases:

- **Popularity Bias:** Due to dataset filtering (≥1000 ratings), the model exhibits popularity bias, frequently recommending blockbusters while failing to suggest highly relevant but niche indie films.
- **Predicted Rating Saturation:** Several items receive a predicted rating of exactly 5.0, making tie-breaking in the Top-10 ranking arbitrary for those positions.
- **Cold-Start:** The model cannot generate recommendations for new users or movies not present in the training set, as their embeddings are never learned.

8. Key Insights

- **Complexity vs. Accuracy Tradeoff:** The computationally simpler GMF model (dot-product) outperformed the highly complex Deep NCF (MLP) in both RMSE and MAP@10. In sparse recommendation datasets, simpler linear combinations often generalize better than deep neural networks.
- **Practical Usability:** GMF is highly practical for production deployment. The learned 128d embeddings can be loaded into an approximate nearest neighbor (ANN) vector database (like FAISS) to serve real-time recommendations to millions of users in milliseconds. Conversely, Deep NCF requires an expensive forward-pass for every prediction.
- **Sparsity as a Feature:** Even at 88.32% sparsity, the model learned meaningful embeddings by concentrating the training signal on reliable, high-activity users and popular movies through intelligent filtering.
- **Metric Alignment:** GMF achieving the best score on both RMSE and MAP@10 simultaneously confirms its suitability as a production recommendation engine — it predicts ratings accurately and ranks recommendations well.

9. Future Improvements

- **Cold-Start Mitigation:** The current architecture strictly relies on collaborative filtering, making it unable to recommend movies to brand new users. Integrating content-based features (movie genres, director, user age) into a Hybrid Architecture would solve this.
- **Temporal Dynamics:** Incorporating the Date feature to model shifting user tastes over time using RNNs or Transformers (e.g., SASRec) could further reduce the RMSE.
- **Diversity-Aware Re-ranking:** A post-processing step (e.g., MMR — Maximal Marginal Relevance) could reduce popularity bias and surface more diverse, niche recommendations without retraining the model.